Event-Driven Microservices

services that talk through events, and how to keep them understandable

Watch the 4-min video

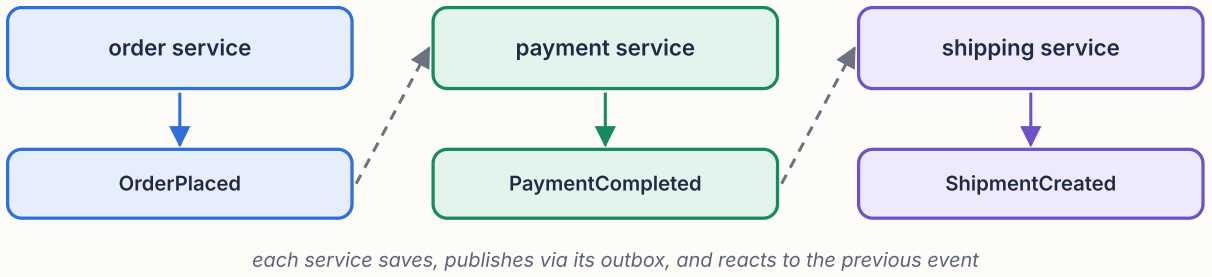
In one line: event-driven microservices publish domain events and react to each other's events, which removes runtime coupling, but needs outboxes, idempotent consumers, correlation IDs, tracing and flow monitoring, or nobody can tell where an order got stuck.

1 Decoupled, and lost

Your system has forty services that talk only through events. Nobody calls anybody directly, so everything is decoupled. Then a customer asks why their order never shipped. Which service dropped it? There's no stack trace across events, no single log, and no one place where the flow is written down.

2 The building blocks

An event-driven microservice owns its data, changes it in local transactions, and publishes **domain events** through an **outbox** (lesson 37). It consumes other services' events with an **inbox** or idempotent handlers (lesson 38), and keeps any data it needs from them as a local copy built from events (event-carried state transfer, lesson 34). Long business flows are **sagas** (lesson 17).



each service saves, publishes via its outbox, and reacts to the previous event

Figure 1. Order to payment to shipping, entirely through events.

WORKED EXAMPLE: ORDER TO PAYMENT TO SHIPPING

The order service saves order 42 and publishes `OrderPlaced` through its outbox, with correlation ID `c-5f1`. The payment service consumes it, charges the card, and publishes `PaymentCompleted` with the same correlation ID. The shipping service consumes that and publishes `ShipmentCreated`. When the customer asks, support searches logs and traces for `c-5f1` and sees the payment event consumed but no shipment: the shipping consumer had sent the event to its DLQ because of an unknown postcode format. A flow monitor that expects `ShipmentCreated` within an hour of `PaymentCompleted` would have alerted first.

3 Making the flow visible

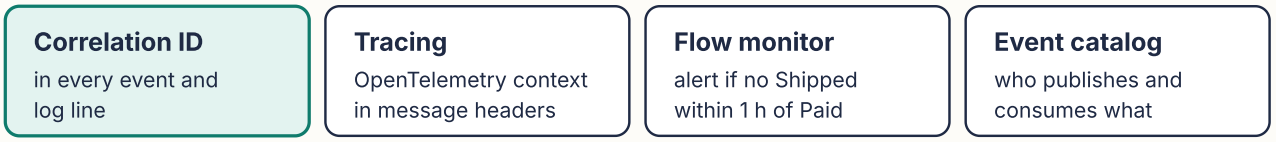


Figure 2. Observability you need before you have forty services.

Pass a **correlation ID** (and causation ID: which event caused this one) in every message and log. Propagate **OpenTelemetry** trace context in message headers so a trace spans services and brokers. Keep an **event catalog** (AsyncAPI, a schema registry) listing producers, consumers and schemas.

4 Choreography or orchestration

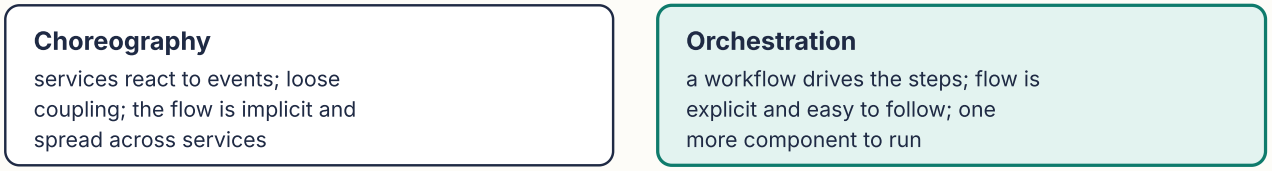


Figure 3. For long or critical flows, an explicit orchestrator is often easier to operate.

COMMON MISTAKES

- No correlation IDs, so flows can't be traced.
- Event chains ten services long with no owner for the whole flow.
- Shared databases between 'event-driven' services.
- Events used for queries that need an immediate answer.

5 Key terms

Domain event: business fact published by the owning service

Causation ID: ID of the message that caused this one

AsyncAPI: specification format for event-driven APIs

Correlation ID: ID shared by all messages of one flow

Event catalog: registry of events, producers and consumers

6 Interview questions

Q1. What are the benefits of event-driven microservices? BASIC

Services don't depend on each other being up, can be added without changing producers, scale independently, and keep their own data.

Q2. What makes them hard to operate? BASIC

Flows are implicit and spread across services, failures are silent, data is eventually consistent, and debugging needs tracing across brokers.

Q3. An order is paid but never shipped. How do you find out why? SCENARIO

Search by correlation ID in logs and traces, check the shipping consumer's lag and DLQ, and add a flow monitor that alerts when a step doesn't follow within its expected time.

Q4. When would you add an orchestrator to a choreographed flow? SCENARIO

When the flow is long, business-critical, changes often, or needs timeouts and compensations that are hard to see across services.

Q5. How do you avoid an event-driven 'distributed monolith'? TRICKY

Keep services owning their data, design events around business facts, avoid synchronous call chains disguised as events, and version event contracts.

Q6. How do traces cross a message broker? TRICKY

The producer puts trace context (W3C traceparent) into message headers; the consumer extracts it and starts a linked span.

Next: SD-I-019 · Cloud Messaging Services backendroadmap.com